Documentação do Solver Transiente de Richards

Pipeline Richards v3 – Aduela de Concreto

1 de dezembro de 2025

1 Enunciado do Problema

Modela-se o fluxo variavelmente saturado no entorno de uma galeria de drenagem profunda (aduela) em regime transiente. A geometria bidimensional é discretizada por elementos triangulares obtidos do arquivo `malha extunderscore dominio.xdmf`. O objetivo é reproduzir a evolução da carga hidráulica h em sete camadas de solo, incorporando a recarga e a extração induzidas pelo nível de água no canal interno.

2 Governo Físico

A formulação parte da Equação de Richards na forma mista (pressão extitheads):

$$C(h)\frac{\partial h}{\partial t} = \nabla \cdot \big[\mathbf{K}(h)(\nabla h + \mathbf{e}_z)\big], \tag{1}$$

onde $C(h) = \partial\theta/\partial h$ é a capacidade hídrica e $\mathbf{K}(h)$ é o tensor de condutividade hidráulica dependente de h.

2.1 Modelo de Van Genuchten–Mualem

Cada material listado em `base.py` possui parâmetros $(\theta_r, \theta_s, \alpha, n, K_s)$, com $m = 1 - 1/n$. As funções constitutivas usadas no código são:

$$heta(h) = \theta_r + (\theta_s - \theta_r)\left[1 + (\alpha|h|)^n\right]^{-m}, \tag{2}$$

$$K(h) = K_s\, S_e^{1/2}\left[1 - \left(1 - S_e^{1/m}\right)^m\right]^2, \tag{3}$$

onde $S_e = (\theta - \theta_r)/(\theta_s - \theta_r)$. A derivada numérica central implementada no solver computa $C(h)$ via

$$C(h) = \max\left(\frac{\theta(h + \delta h) - \theta(h - \delta h)}{2\,\delta h},\ 10^{-8}\right), \qquad \delta h = 10^{-4}\ \text{m}. \tag{4}$$

3 Formulação Fraca e Elementos Finitos

Adotando funções de peso w, integra-se a equação em um elemento Ω_e e aplica-se integração por partes, obtendo-se

$$\int_{\Omega_e} w\, C(h) \frac{\partial h}{\partial t}\, d\Omega + \int_{\Omega_e} \nabla w^T \mathbf{K}(h)(\nabla h + \mathbf{e}_z)\, d\Omega = \int_{\Gamma_q} w\, q_n\, d\Gamma. \tag{5}$$

Usam-se elementos triangulares lineares P_1. As funções de forma N_i conduzem ao vetor gradiente constante por elemento, com a matriz

$$\mathbf{B} = \frac{1}{2A_e} \begin{bmatrix} b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}, \tag{6}$$

onde A_e é a área, $b_i = y_j - y_k$ e $c_i = x_k - x_j$. O solver monta:

$$\mathbf{A}_e = K(h_e) A_e\, \mathbf{B}^T \mathbf{B}, \tag{7}$$

$$\mathbf{M}_e = \frac{C(h_e)}{\Delta t} \frac{A_e}{12} \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \tag{8}$$

$$\mathbf{b}_e^{\text{grav}} = \frac{K(h_e)}{2} \begin{bmatrix} c_1 & c_2 & c_3 \end{bmatrix}^T, \tag{9}$$

com $h_e = \frac{1}{3}\sum h_i$ representando a média nodal do elemento.

4 Discretização Temporal e Iterações de Picard

O termo transiente é integrado por Backward Euler com $\Delta t = 600$ s (10 minutos). Em cada passo $n \to n+1$ resolve-se

$$\left(\mathbf{M}(h^{(k)}) + \mathbf{A}(h^{(k)})\right) \Delta h^{(k)} = \mathbf{M}(h^{(k)}) h^n - \mathbf{b}^{\text{grav}}(h^{(k)}) - \mathbf{A}(h^{(k)}) h^{(k)}, \tag{10}$$

utilizando iterações de Picard, onde $h^{(k+1)} = h^{(k)} + \Delta h^{(k)}$. A convergência é avaliada pelo erro relativo

$$\varepsilon = \frac{\| h^{(k+1)} - h^{(k)} \|_2}{\| h^{(k+1)} \|_2 + 10^{-12}} < 10^{-4}. \tag{11}$$

O número máximo de iterações é 15. O horizonte temporal padrão cobre 24 h (144 passos). A cada 36 passos (6 h) o campo de pressão é salvo para pós-processamento.

5 Condições de Contorno e Inicial

O domínio possui fronteiras fixas descritas diretamente no código:

- Topo ($y = y_{\max}$): $h = -2$ m, representando uma superfície não saturada.
- Fundo ($y = y_{\min}$): $h = 0$ m.
- Esquerda ($x = x_{\min}$): $h = 0$ m (lençol freático alto).
- Direita ($x = x_{\max}$): $h = -2$ m (lençol baixo).

Todos os nós com essas condições são tratados por imposição direta (linha nula, diagonal unitária) após montar o sistema em formato LIL, convertendo para CSR somente antes do `spsolve`. A condição inicial é seca: $h = -10$ m em todo o domínio, exceto nos nós onde as condições de Dirichlet já são aplicadas.

5.1 Mapeamento das Tags da Malha

O arquivo `malha_tubular.geo` (Gmsh) atribui etiquetas físicas que são preservadas no par `malha_dominio.xdmf/h5` e lidas pelo solver. A tabela a seguir relaciona cada tag ao seu papel no modelo:

oprule Entidade	`Nome no Gmsh`	Uso no Solver
Superfície	`Solo_Inferior`	Região com material profundo (definições em `base.py`).
Superfície	`Solo_Superior`	Região com propriedades da camada superior.
Superfície	`Envelope_Brita`	Camada intermediária de alta condutividade.
Superfície	`Concreto`	Domínio sólido da aduela (apenas para visualização).
Curva	`Contorno_Externo`	Conjunto de fronteiras retangulares; o solver identifica os nós associados e aplica as cargas fixas citadas acima conforme a coordenada (x ou y).
Curva	`Parede_Interna_Rio`	Limite interno correspondente à parede da galeria; atualmente recebe a mesma regra Dirichlet definida por coordenadas, podendo ser usada futuramente para condições dependentes do nível d'água.
Ponto	`Piezo_Longe`	Nó monitorado distante da aduela.
Ponto	`Piezo_Perto`	Nó monitorado próximo à parede externa.
Ponto	`Piezo_Fundo`	Nó monitorado sob a aduela.

Os piezômetros são pontos forçados na malha 2D para extrair séries temporais consistentes mesmo quando a malha é refinada. O nó `Piezo_Longe` registra o comportamento do lençol freático longe da interferência da galeria, `Piezo_Perto` acompanha a zona de maior gradiente hidráulico junto à parede, e `Piezo_Fundo` monitora possíveis subpressões imediatamente abaixo da aduela. Esses índices são lidos diretamente após importar o arquivo `.xdmf`, permitindo que o solver armazene automaticamente a evolução de $h(t)$ em cada posição.

As curvas físicas servem como "guarda-chuvas" para o pós-processamento: ao ler os datasets de fronteira em `solver_richards_transiente.py`, obtêm-se os índices dos nós para cada segmento. Contudo, a versão atual simplifica a aplicação das condições pela inspeção direta das coordenadas, uma vez que as faces externas são alinhadas aos eixos. Ainda assim, manter as tags facilita evoluções como aplicar fluxos distintos por fronteira ou ativar acoplamento rio-solo apenas na curva `Parede_Interna_Rio`. As tags de pontos garantem que os mesmos nós existam independentemente do refinamento de malha, preservando séries temporais consistentes nos piezômetros.

6 Algoritmo Implementado

O arquivo `solver_richards_transiente.py` executa os passos:

1. Ler malha `.xdmf` e tags de material via `h5py`.

2. Construir listas de nós de contorno, definir Δt, T e tolerâncias.
3. Para cada passo temporal:
   (a) Inicializar $h^{(0)} = h^n$.
   (b) Montar matrizes globais com $h^{(k)}$.
   (c) Aplicar condições de Dirichlet no formato LIL e converter para CSR.
   (d) Resolver com `spsolve` e verificar a convergência de Picard.
4. Armazenar campos selecionados e escrever `VTU` e o arquivo coleção `PVD` para o ParaView.
5. Registrar estatísticas finais (mínimo, máximo, média de h) e status de convergência.

7 Saídas e Diagnósticos

Durante a simulação, o solver imprime para cada passo relevante:

- Faixa de $C(h)$ e $K(h)$ (diagnóstico de estabilidade).
- Número de iterações de Picard e erro relativo.
- Alertas caso o critério de convergência não seja atendido.

O pós-processamento gera `drenagem_# .vtu` e uma coleção `drenagem.pvd`. Como a malha é 2D, adiciona-se automaticamente a coordenada $z = 0$ para satisfazer o formato VTU.

8 Parâmetros Numéricos Principais

- Passo de tempo: $\Delta t = 600$ s.
- Horizonte simulado: 24 h.
- Critério de Picard: $\varepsilon < 10^{-4}$ com no máximo 15 iterações.
- Formato de matrizes: LIL para montagem/BC, CSR para `spsolve`.
- Exportação: resultados a cada 6 h para limitar o tamanho do `PVD`.

9 Considerações Finais

O workflow descrito combina leitura de malha `meshio`, manipulação de `h5py`, montagens em `SciPy sparse` e diagnóstico em tempo real. A técnica de Picard associada ao refinamento temporal de 10 minutos permitiu estabilizar o comportamento transiente observado experimentalmente, fornecendo um relatório textual colorido e arquivos VTU prontos para inspeção no ParaView.